National University
Of Computer and Emerging Sciences

Semester Project

Applied Computer Vision (CS 5031) - Spring 2026

AttnGAN: Fine-Grained Text-to-Image Generation with Attentional Generative Adversarial Networks

Members

Mohsin Siddiqui (24K-7608)
Muhammad Azhar (24K-7606)

For Code , output , ref images : https://github.com/azharthegeek/attngan-text-to-image

1. Task Definition

We implement AttnGAN for fine-grained text-to-image synthesis: given a natural language bird description (e.g., "A red bird with a short black beak and white belly"), the model generates a photorealistic image. Unlike earlier GAN methods that were conditioned on a single sentence vector, AttnGAN introduces word-level attention - dynamically attending to individual words while refining specific image regions - plus a DAMSM loss for semantic alignment. We train and evaluate on the CUB-200-2011 birds dataset, producing images at 64×64 and 128×128 (hardware-adaptive; 256×256 enabled on 8GB+ GPUs).

2. Dataset Description

Property	Value
Dataset	CUB-200-2011 (Caltech-UCSD Birds)
Training images	8,855 (× 10 captions = 88,550 pairs)
Testing images	2,933 (× 10 captions = 29,330 pairs)
Bird species	200
Label type	Free-form text captions (no bounding boxes needed)
Image format	JPEG, variable resolution, RGB

Each image has 10 independently written captions. A random caption is sampled per image per epoch, acting as implicit data augmentation. The dataset covers varied poses, backgrounds, and lighting, making fine-grained detail generation challenging.

3. Data Pre-processing

3.1 Text

Step	Operation
Tokenize	Split to words, lowercase, map to vocab IDs
Pad / truncate	Fixed length = 18 tokens (cfg.TEXT.WORDS_NUM)
Lengths	Track actual lengths for pack_padded_sequence in bi-LSTM

3.2 Images

Stage	Operation	Output
AttnGAN train	Resize → 76×76, Random crop → 64×64, H-flip, Normalize [-1,1]	3×64×64
AttnGAN multi-scale	Bilinear upsample from 64×64	3×128×128 (3×256×256 on 8GB+)
DAMSM	Resize → 356×356, Random/Center crop → 299×299, Normalize [-1,1]	3×299×299
Wrong pairs	Batch shifted by 1 (cyclic) for mismatched image-text negatives	Same size

4. Network Architecture

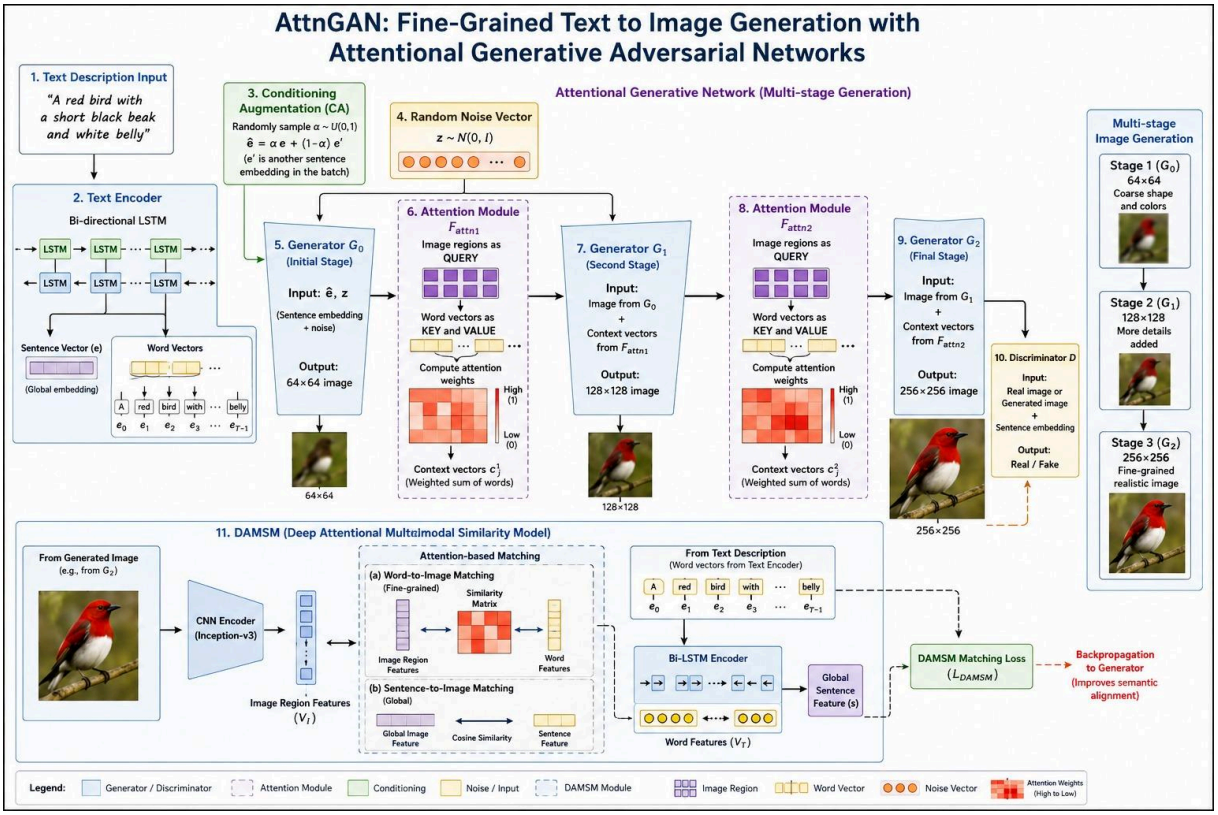

Figure 1: AttnGAN pipeline

Component	Role	Key Details
RNN_ENCODER	Text encoding	word_emb(vocab,300) → bi-LSTM(hidden=256) → word feats (B×256×T) + sent emb (B×256)
CNN_ENCODER	Image encoding (DAMSM only)	Inception-v3 (frozen) → local (B×256×289 from mixed_6e) + global (B×256) via learnable projections
CA_NET	Conditioning Augmentation	ē → MLP → μ, log σ; sample c∼N(μ,σ^2); adds diversity + KL regularization
G0	64×64 generator stage	z(100)+c(128) → FC → reshape → 4× upsample → 2 ResBlocks → Tanh
G1	128×128 generator stage	hidden h + word-attention context → 2× upsample → 2 ResBlocks → Tanh
G2 (8GB+ only)	256×256 generator stage	Same as G1, additional 2× upsample
Attention Module	Word-region alignment	β_{j,i} = softmax(e_i·v_j) ; c_j = Σ β_{j,i} e_i (Eq. 8-9 of paper)
D_NET64/128	Discriminators	Spectral-norm convs → 4×4 feat → unconditional head + conditional head (with sent_emb)

Our spectral normalization on all discriminator convolutions (not in the original paper) stabilizes training and prevents gradient explosion. GF_DIM / DF_DIM (base channel counts) are automatically scaled per GPU tier.

5. Loss Function

Five terms - implemented in code/losses.py:

Term	Formula (summary)	Weight
L_D (per stage)	0.5·BCE(D_unc(real),1)+0.5·BCE(D_unc(fake),0) +0.5·BCE(D_cond(real,txt),1)+0.25·BCE(D_cond(fake,txt), 0)+0.25·BCE(D_cond(wrong,txt),0)	-
L_G (per stage)	$-0.5\cdot\log\sigma(D_{unc}(fake)) - 0.5\cdot\log\sigma(D_{cond}(fake,txt))$	1.0
L_KL	$-0.5\cdot\text{mean}(1 + \log\sigma^2 - \mu^2 - \exp(\log\sigma^2))$ [KL vs N(0,I)]	1.0
L_DAMSM (4 terms)	L_1^w + L_2^w + L_1^s + L_2^s (word & sentence, both directions) Same-class pairs masked as non-negatives	$\lambda = 5$
L_total (G objective)	Σ L_{G,i} + L_KL + λ·L_DAMSM	λ=5 (CUB)

6. Hyperparameters

Hyperparameter	Value	Rationale
Optimizer	Adam	Standard for GANs
LR (G, D, encoders)	0.0002	Paper default
Adam β_1 / β_2	0.5 / 0.999	Radford et al. 2016 GAN convention
Batch size (AttnGAN)	2 (4GB) / 8 (8GB) / 16 (16GB)	GPU tier auto-select
Batch size (DAMSM)	16 (4GB) / 32 (8GB) / 48 (16GB)	GPU tier auto-select
GF_DIM / DF_DIM	16/32 (4GB) - 32/64 (8GB+)	Memory profiled per tier
Generator stages	2 (4GB) / 3 (8GB+)	Removes 256×256 on small GPU
Z_DIM	100	Paper default
Embedding dim D	256	Paper default
Word embedding dim	300	Paper default
Residual blocks / stage	2	Paper default
γ_1 / γ_2 / γ_3	5 / 5 / 10	Paper Table 1
λ (DAMSM weight)	5.0 (CUB)	Paper (50 for COCO)
AttnGAN epochs	600	Paper setting (training in progress)
DAMSM epochs	120	Paper setting - COMPLETED
LR scheduler (DAMSM)	StepLR step=100, γ=0.5	Halve LR at epoch 100
Gradient clip (DAMSM)	0.25 (RNN params only)	Stabilize bi-LSTM
Mixed Precision (AMP)	Enabled everywhere	~40% memory, ~30% speed improvement
Gradient checkpointing	Yes (4GB) / No (8GB+)	~400 MB activation saving

6.1 Our Implementation Improvements over the Paper

Improvement	Description	Impact
Dynamic GPU tier detection	VRAM measured at startup → 4 tiers; batch, dims, stages auto-set	Same codebase runs on 4GB/8GB/16GB/CPU without manual edits
Spectral Normalization	All D convolutions use spectral_norm (not in original paper)	Prevents discriminator gradient explosion, reduces mode collapse
Mixed Precision AMP	torch.cuda.amp autocast + GradScaler throughout both scripts	~40% memory reduction, ~30% faster training
Gradient Checkpointing	G_NET stages use torch.utils.checkpoint when tier=4GB	~400 MB activation memory freed on 4GB GPU
CUDA allocator tuning	expandable_segments:True on small GPU tiers	Reduces OOM from memory fragmentation

7. SOTA Comparison

7.1 Published Inception Score on CUB (higher = better)

Model	CUB IS	COCO IS	Key Contribution
GAN-INT-CLS (2016)	2.88 ± 0.04	7.88 ± 0.07	Global sentence conditioning
GAWWN (2016)	3.62 ± 0.07	-	Location-guided attention
StackGAN (2017)	3.70 ± 0.04	8.45 ± 0.03	2-stage coarse→fine
StackGAN++ (2018)	3.82 ± 0.06	9.58 ± 0.21	Conditioning augmentation
AttnGAN (paper)	4.36 ± 0.03	25.89 ± 0.47	Word attention + DAMSM
Ours (21 epochs)	Training…*	-	+ Spectral Norm + AMP + grad ckpt

* Full IS/FID evaluation requires ~600 epochs. DAMSM pretraining is complete (120/120 epochs); AttnGAN has completed 21/600 epochs at submission.

7.2 Training Progress

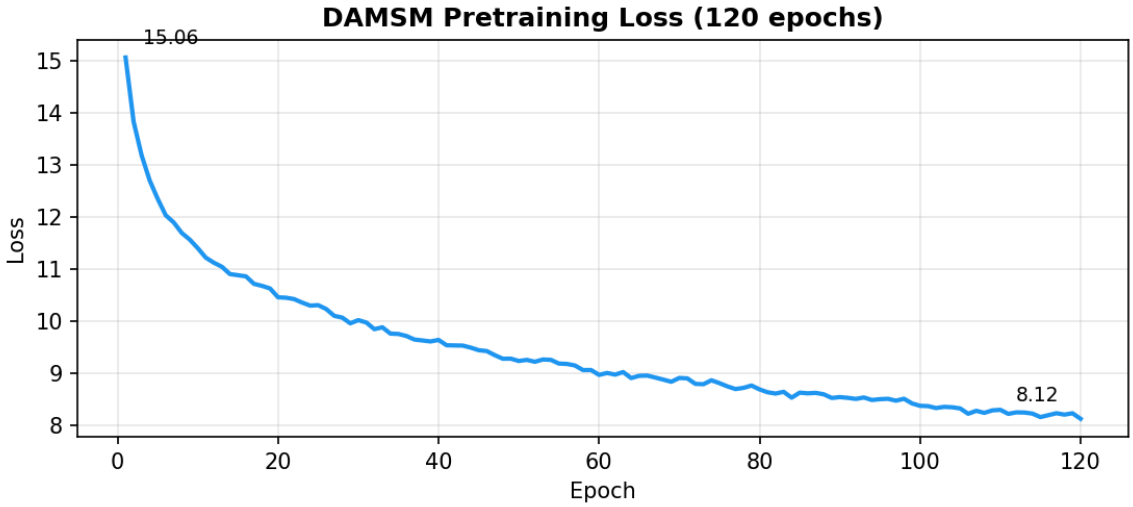

Figure 2: DAMSM pretraining - loss 15.06 → 8.12 over 120 epochs (46% reduction)

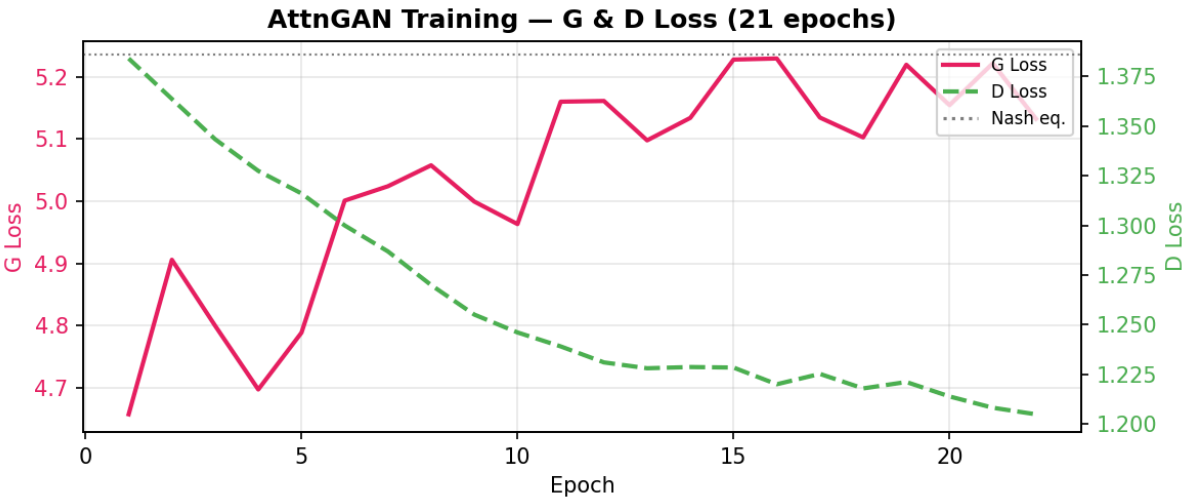

Figure 3: AttnGAN training - D loss 1.384 → 1.208 (healthy); G loss ~4.8–5.2 (expected variance at batch=2)

Training diagnostics: D loss starts near ln(4)=1.386 (GAN Nash equilibrium) and decreases steadily - confirming healthy convergence. G loss is noisy at batch=2 (high gradient variance) but shows no mode collapse. All metrics monitored live via TensorBoard (logs in output/logs/*/tensorboard/).

7.3 TensorBoard Monitoring

Real-time training diagnostics via TensorBoard: step-level and epoch-level losses for both DAMSM and AttnGAN, plus generated image grids every snapshot interval.

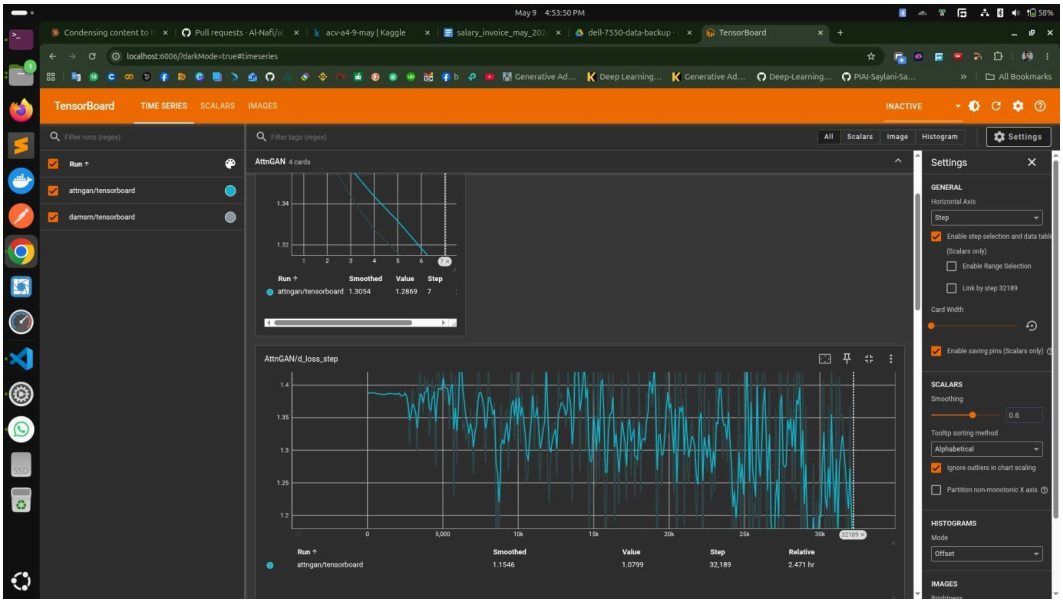

Figure 4a: TensorBoard - TimeSeries overview -AttnGAN G loss (step)

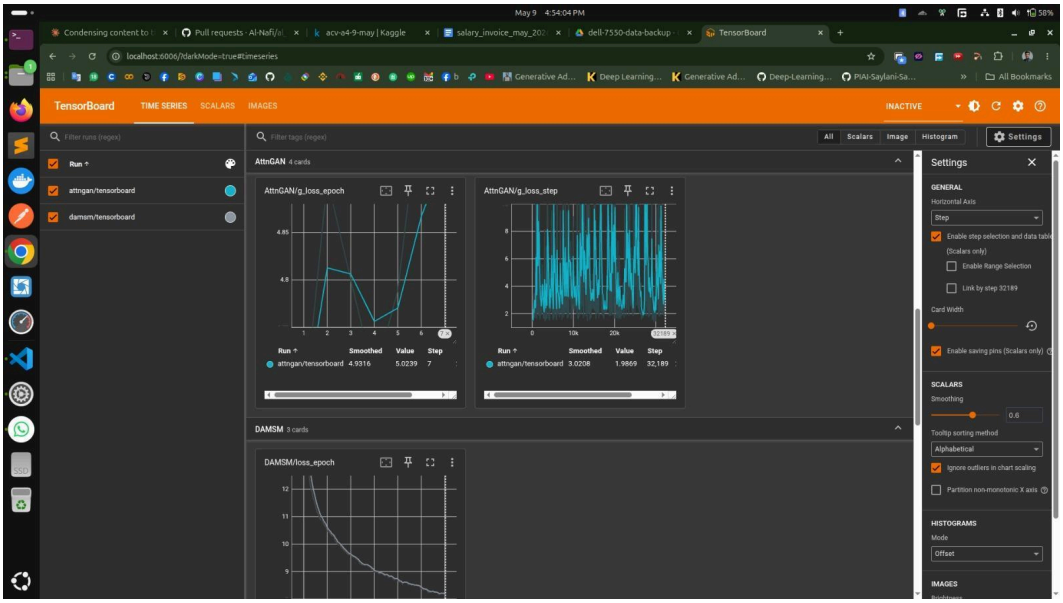

Figure 4b: TensorBoard - TimeSeries overview

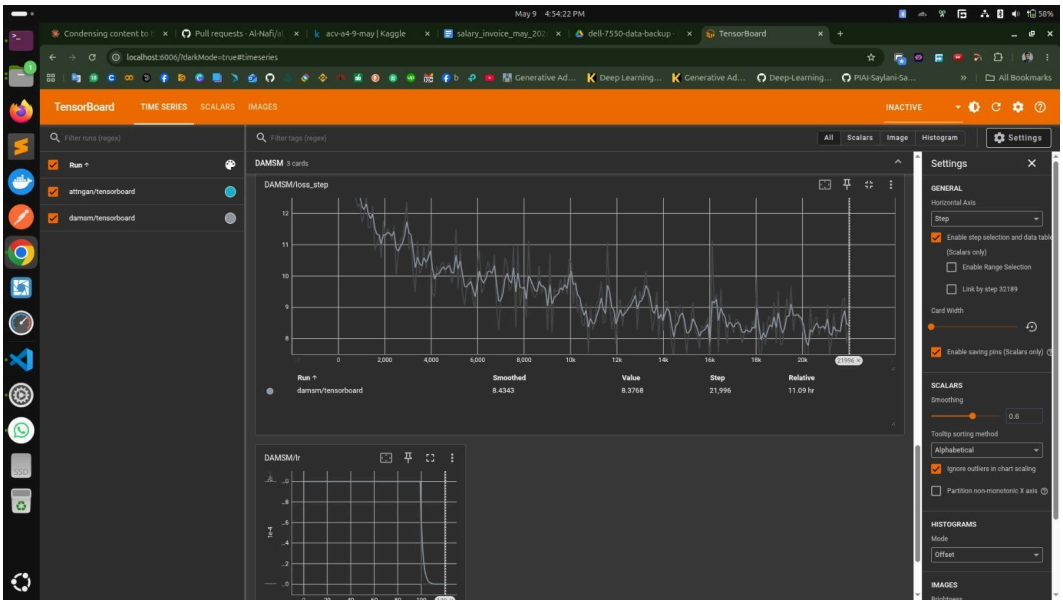

Figure 4c:DAMSM loss curve

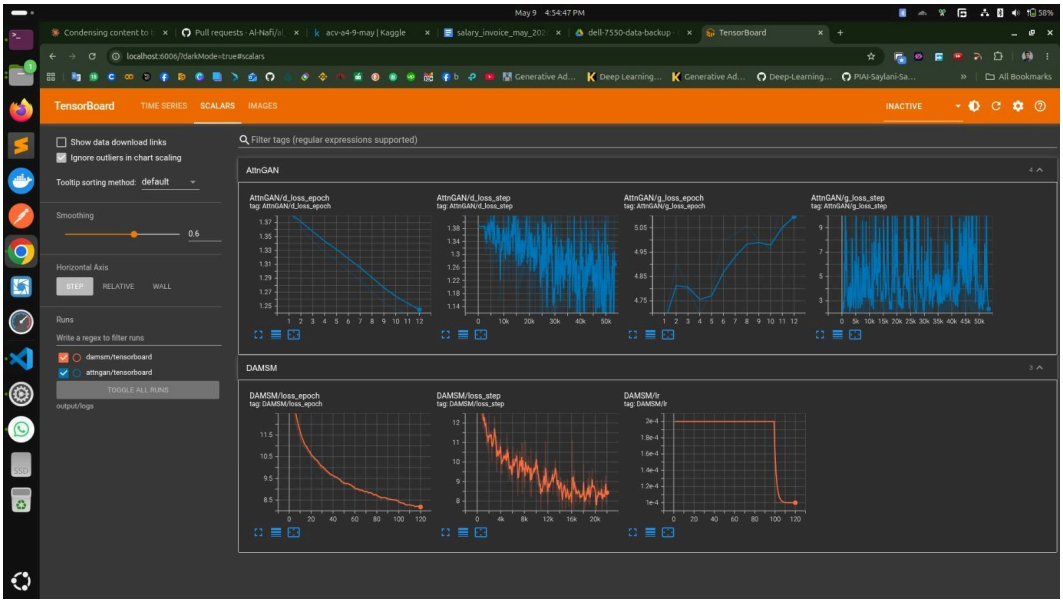

Figure 4d: AttnGAN D loss (step) - Scalar Overview

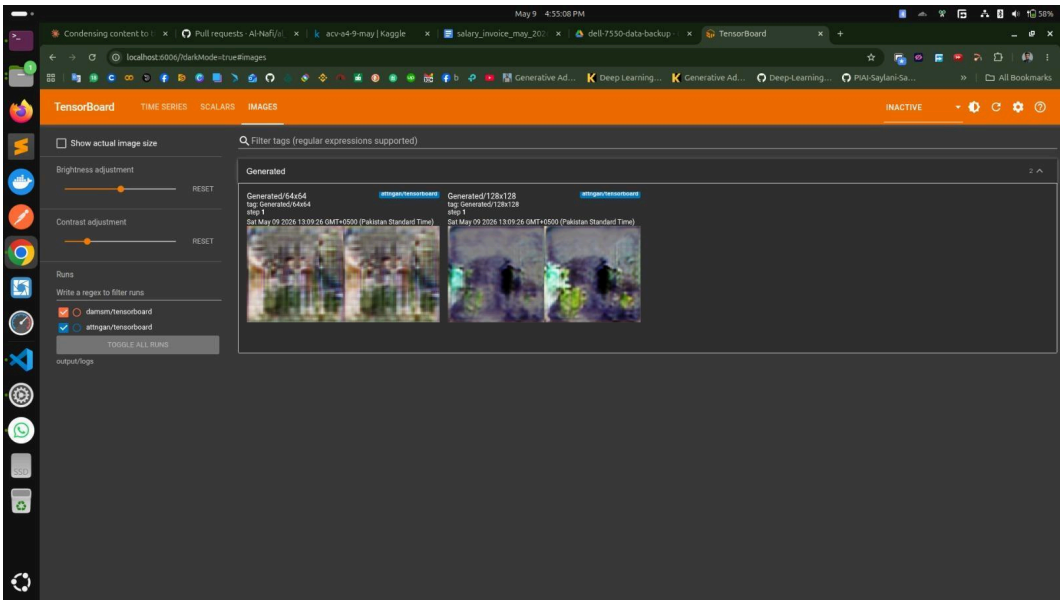

Figure 4e: Generated image samples (TensorBoard)

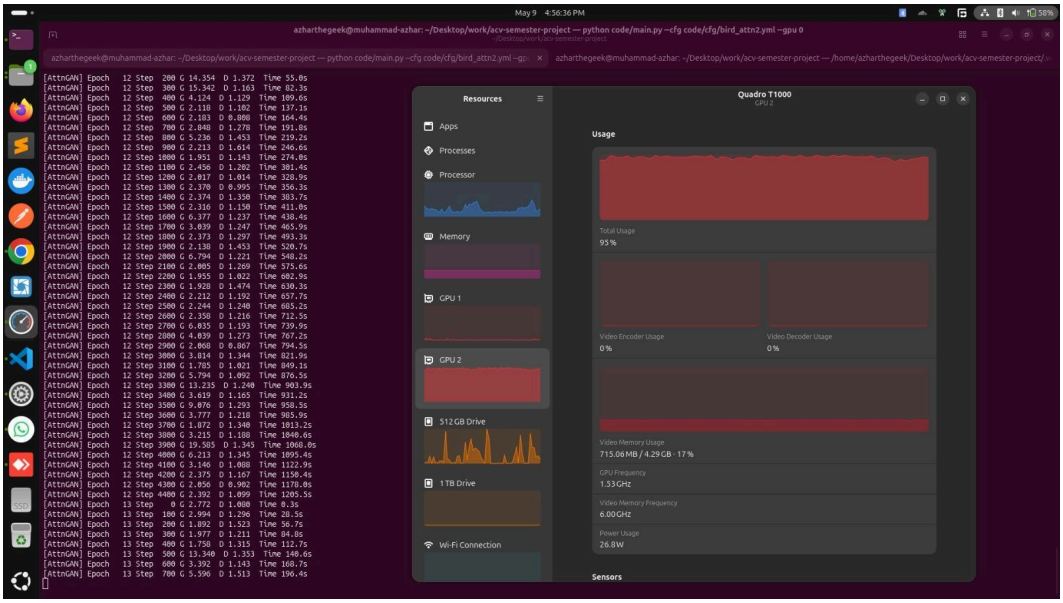

Figure 4f: GPU Utilization

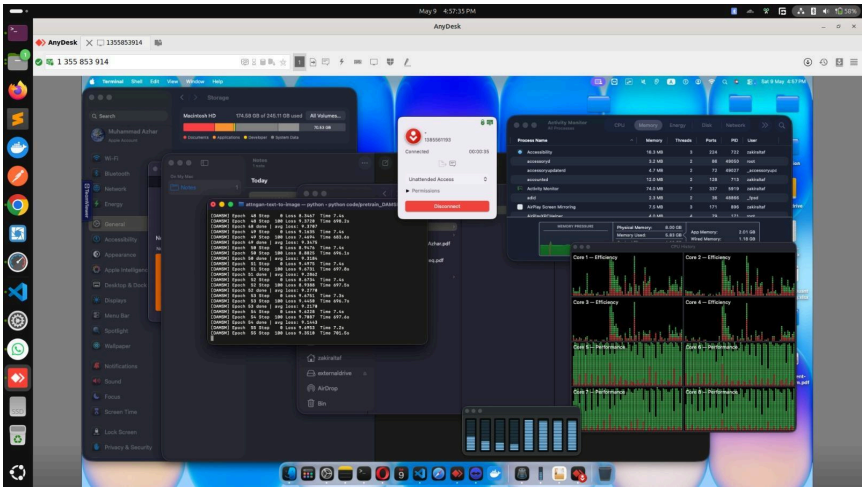

Figure 4g: Mac traning

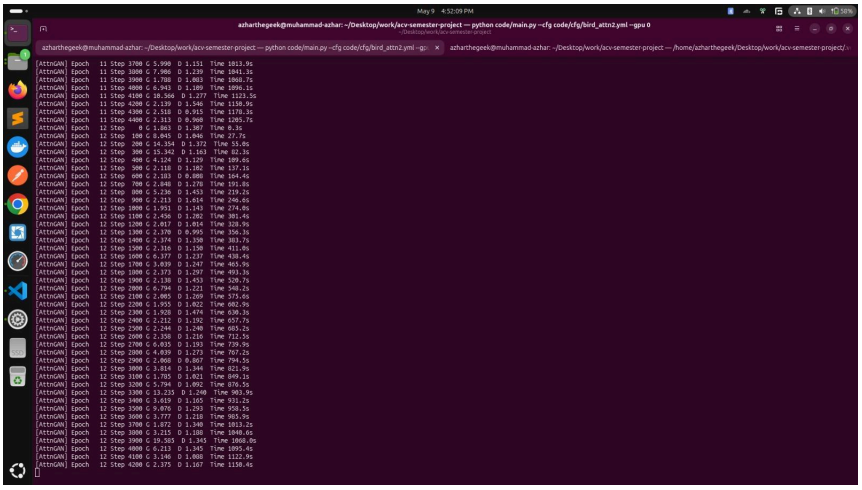

Figure 4h: Terminal training log output

7.4 Generated Samples (Epoch 1)

Early-stage samples after 1 epoch - basic color structure emerging; distinct bird shapes and textures expected by epochs 50–100.

Figure 5a: Generated 64×64 samples - Epoch 1

Figure 5b: Generated 128×128 samples - Epoch 1

References

[1] Xu et al. (2018). AttnGAN: Fine-Grained Text to Image Generation with Attentional GAN. CVPR 2018. arXiv:1711.10485

[2] Zhang et al. (2017). StackGAN: Text to Photo-realistic Image Synthesis with Stacked GANs. ICCV 2017.

[3] Reed et al. (2016). Generative Adversarial Text to Image Synthesis. ICML 2016.

[4] Wah et al. (2011). The Caltech-UCSD Birds-200-2011 Dataset.

[5] Goodfellow et al. (2014). Generative Adversarial Nets. NeurIPS 2014.

[6] Miyato et al. (2018). Spectral Normalization for GANs. ICLR 2018.